Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Dersin tarihi 25 Ekim 2021 Pazartesi. Ders çevrim içi yapıldı.
  - Asıl anlatım yaklaşık 00:05'te başlar. İlk dakikalar katılımcı bekleme ve sorulardır.
  - Hoca bu derste az ilerleyeceğini söyledi. Ertesi gün diğer grupların dersini yapamayacağı için grupları dengelemek istiyor. Ders yaklaşık 01:20'de bitiyor.
  - Slaytlar Silberschatz, Galvin ve Gagne, *Operating System Concepts*, 9. baskı (karelerden).
- İşletim sistemi yapıları (Bölüm 2'nin sonu)
  - MS-DOS: tek görevli (single tasking), basit yapı.
  - Komut yorumlayıcısının (command interpreter) yerleşik (resident) ve geçici (transient) kısımları.
  - Geleneksel UNIX yapısı: sistem programları ve monolitik çekirdek (kernel).
  - Unix'te `fork` ve `exec` ile program çalıştırma, çıkış kodu (return code) geleneği.
  - Sistem programları ve işletim sisteminin gerçeklenme dili (Assembly, C, C++).
  - Katmanlı yaklaşım (layered approach).
  - Mikroçekirdek (microkernel) ve mesajlaşma maliyeti.
  - Yüklenebilir çekirdek modülleri (loadable kernel modules): `insmod`, `rmmod`, `modprobe`.
  - Karma (hybrid) yapılar: Linux, Windows, Mac OS X.
- Bölüm 3: Süreçler (processes)
  - Süreç kavramı: çalışan program. Job, task ve process terimlerinin birbirinin yerine kullanılması.
  - Bellekte süreç: text, data, heap ve stack bölümleri.
  - Süreç durumları (process states): new, ready, running, waiting, terminated.
  - Süreç kontrol bloğu (PCB, Process Control Block).
  - İki süreç arasında bağlam değişimi (context switch) örneği.
  - İş parçacıklarına (threads) kısa giriş.
  - Süreç zamanlama kuyrukları (scheduling queues): job, ready ve aygıt kuyrukları.
  - Kısa, uzun ve orta vadeli zamanlayıcılar (short-term, long-term, medium-term scheduler), takas (swapping).
- Ders slayt 3.18'de bitti.

Hocanın Özellikle Vurguladığı Kısımlar

- 2.45-2.46 sonrası slaytlar (işletim sistemi yapılarının devamı) derste anlatılmadı.
  - Bu kısımdan da sorumlusunuz. Kitaptan ve slaytlardan okuyun.
- Job, task ve process aynı kavrama karşılık gelir.
  - Terimler işletim sistemlerinin gelişim sürecinden kalmadır. Derste çoğunlukla "process" kullanılacak.
- Süreç ve program farkını sınıf ve nesne (class-object) ilişkisine benzetin.
  - Program diskte duran komutlar bütünüdür. Süreç onun bellekte çalışan hâlidir.
- Terminated durumu neden var?
  - Süreç bitince çıkış bilgisi kaybolmasın. Ebeveyn süreç (parent) çocuğun (child) nasıl bittiğini öğrenebilsin.
- Unix'te başarılı biten süreç 0 döndürür. Hata varsa 0'dan farklı bir değer döner.
  - Kabuk (shell) bir komutun başarılı olup olmadığını bu değerden anlar.
- Mikroçekirdekte her mesaj kullanıcı modundan çekirdek moduna ve geri geçiş demektir.

  - Bu ek yük performansı düşürür. Modül yapısı bu sorunu azaltmak için ortaya çıktı.
- Güncel işletim sistemlerinin hiçbiri tek bir mimariyle yapılmaz.
  - Linux büyük ölçüde modüler ama monolitik kısmı da var. Windows büyük ölçüde monolitik, bir kısmı mikroçekirdeğe benzer. Mac OS X katmanlı ve modüler.
- Bellekte süreç resmine dikkat edin: altta 0, üstte max yazar.
  - Her süreç belleğin tamamı kendisininmiş gibi davranır. Arka planda yalnızca kullandığı segmentler fiziksel belleğe yerleştirilir. Bu konu bellek yönetiminde açılacak.
- Kuyruk diyagramı slaytındaki "fork" kısmı yanıltıcı olabilir.
  - Grafik, çocuk bitene kadar ebeveynin beklediğini ima eder. Fork sonrası ikisi de ready kuyruğuna girer, hangisinin çalışacağına zamanlayıcı karar verir.
- Uzun vadeli zamanlayıcı G/Ç ağırlıklı (I/O bound) ve CPU ağırlıklı (CPU bound) süreçleri dengeli seçmelidir.
- **Sınav biçimi** (dersin sonunda bir soruya cevap):
  - Sınavlar çoktan çok seçmeli (multiple-answer) olacak. Bir sorunun birden fazla şıkkı doğru olabilir.
  - Sorular büyük ihtimalle İngilizce olacak. Hoca Türkçede kavramların kayabildiğini söyledi.
  - Yanlışlar doğruyu götürür: 4 şıkta 3 yanlış 1 doğruyu, 5 şıkta 4 yanlış 1 doğruyu götürür.
  - Soru sayısı belli değil. Vize 8. ya da 9. hafta olabilir; ilk hafta çevrim içi ders yapılamadığı için kesin değil.
- **Uygulamalar (laboratuvar):**
  - Bu hafta başlamayacak. Asistanlarla takvim bu hafta planlanacak, büyük ihtimalle sonraki hafta başlayacak.
  - Uygulamalar çevrim içi ve kayıtlı olacak. Gerekçe: grupların toplamı 200'e yakın, laboratuvara sığmıyor; kayıttan tekrar izlenebiliyor.
  - Duyurular OBS üzerinden ve bölüm web sayfasından yapılacak.

Kısa Tekrar Notları

- MS-DOS: tek görev, bellek yönetimi yok, süreç kavramı neredeyse yok.
- Komut yorumlayıcısı: yerleşik kısım her zaman bellekte. Geçici kısım program çalışırken bellekten çıkarılabilir, sonra yerleşik kısım tarafından geri yüklenir.
- MS-DOS'ta uygulamalar BIOS sürücülerine doğrudan erişebiliyordu.
- UNIX: çok kullanıcılı, çok görevli (time sharing). Çekirdek büyük ve monolitik.
- Monolitik çekirdeğin sorunu: büyük bellek tüketimi ve yavaşlık.
- Katmanlı yapı: her katman yalnızca alt katmandan hizmet alır.
- Mikroçekirdek: çekirdekte yalnızca temel hizmetler (bellek yönetimi, CPU zamanlama, süreçler arası iletişim). Geri kalan her şey kullanıcı modunda. Haberleşme çekirdek üzerinden mesajla.
- Yüklenebilir çekirdek modülü: ihtiyaç olunca yüklenir (`insmod`/`modprobe`), bitince kaldırılır (`rmmod`).
- Süreç = program sayacı (program counter) + yazmaçlar (registers) + text + data + heap + stack + açık dosyalar vb.
- Stack ve heap boyutu değişir. Stack yukarıdan aşağı, heap aşağıdan yukarı büyür.
- 5 durum: new → ready → running → (waiting) → ready → … → terminated.
- PCB'de tutulanlar: süreç kimliği (PID), durum, program sayacı, yazmaçlar, CPU zamanlama bilgisi (öncelik vb.), bellek yönetimi bilgisi, hesap bilgisi (accounting), G/Ç durum bilgisi (açık dosyalar, aygıtlar).
- Linux'ta PCB karşılığı `task_struct`.
- Bağlam değişimi: çalışan sürecin durumu PCB'ye kaydedilir, sonraki sürecin PCB'si yüklenir.
- Çalışacak süreç yoksa Windows'ta "System Idle Process" çalışır.
- Thread: aynı süreçteki kod, data ve heap'i paylaşan, ayrı stack'i olan akış.
- Kuyruklar bağlı liste (linked list) ile gerçeklenir. Her aygıtın ayrı bekleme kuyruğu var.
- Kısa vadeli zamanlayıcı: bir sonraki CPU döngüsünde hangi süreç çalışacak.
- Uzun vadeli zamanlayıcı: diskteki hangi işler belleğe alınacak.
- Orta vadeli zamanlayıcı: takas (swapping) ile süreçleri bellekten çıkarır ve geri alır.

Detaylı Açıklamalar

MS-DOS ve komut yorumlayıcısı

MS-DOS (Disk Operating System) tek görevli bir sistemdir. Aynı anda tek bir program çalışır. Bu yüzden süreçler arası geçiş, bellek paylaşımı ve bellek yönetimi gibi sorunlar yoktur.

Sistemde bir çekirdek ve bir komut yorumlayıcısı vardır. Komut yorumlayıcısı ile program yüklenir ve çalıştırılır. Eskiden bilgisayar operatörünün yaptığı işler bu şekilde kullanıcıya geçmiştir.

Slayttaki iki resim farklıdır:

- Sağdaki resimde boş bellek (free memory) daha azdır, çünkü bir program çalışıyor.
- Sağdaki resimde komut yorumlayıcısı daha küçüktür. Bu bir çizim hatası değildir.

Çalıştırılabilecek programın boyutu boş bellek kadardır. Boş belleği büyütmek için komut yorumlayıcısı iki parçaya ayrılır:

- **Yerleşik (resident) kısım:** her zaman bellektedir.
- **Geçici (transient) kısım:** program çalışırken bellekten kaldırılır, yerini programa bırakır.

Program bitince yerleşik kısım, geçici kısmı diskten yeniden belleğe yükler.

MS-DOS'ta katmanlar arasında sıkı bir ayrım yoktur. Yerleşik sistem programları ve hatta uygulamalar, MS-DOS sürücülerini atlayıp doğrudan BIOS'taki sürücüleri kullanabilir. BIOS her aygıtı desteklemeyebilir. Örnek: CD-ROM'lar için MS-DOS'un kendi sürücüsü vardı. Bu sürücü kesme (interrupt) ile değil yoklama (polling) ile çalışıyordu.

UNIX ve monolitik çekirdek

UNIX çok kullanıcılı ve çok görevlidir. Süreçler arasında zaman paylaşımı (time sharing) yapılır. Bu da bellek yönetimini gerekli kılar.

Geleneksel UNIX iki parçaya ayrılır: sistem programları ve çekirdek. Kabuk, derleyiciler ve sistem kütüphaneleri üstte; aralarında sistem çağrısı arayüzü (system-call interface) vardır. Çekirdekte sinyaller, terminal sürücüleri, dosya sistemi, takas, blok G/Ç, disk sürücüleri, CPU zamanlama, sayfa değiştirme (page replacement), istem üzerine sayfalama (demand paging) ve sanal bellek (virtual memory) bulunur.

Bu kadar işi yapan çekirdek büyüktür. Büyük çekirdeğin sakıncaları:

- Daha fazla bellek kullanır. İşletim sistemi ne kadar az kaynak kullanırsa o kadar iyidir.
- Yüklemesi ve çalışması yavaşlar. Örneğin zamanlama kararının çok kısa sürede verilmesi gerekir. Aksi hâlde CPU, işi yapmak yerine "işi kim yapacak" kararına harcanır.

Kabuk komutları şöyle çalışır:

1. `fork` sistem çağrısı ile yeni bir süreç yaratılır.
2. `exec` sistem çağrısı ile istenen program bu sürecin içine yüklenir.
3. Süreç bitince bir çıkış kodu üretir. Gelenek: başarı 0, hata 0'dan farklı.

İşletim sistemi hangi dille yazılır?

İlk işletim sistemleri Assembly ile yazılırdı. ALGOL ve PL/I gibi üst seviye diller de kullanıldı. UNIX'i geliştiren ekip C dilini de geliştirdi. C'nin amacı, Assembly'den daha esnek ve taşınabilir (portable) bir dille işletim sistemi yazmaktı.

Bugün işletim sistemlerinin çoğu C ve C++ ile yazılır. Önyükleyici (bootloader) ve çekirdeğin bazı kısımları hâlâ Assembly'dir. Sistem programlarında Perl, Python ve kabuk betikleri (shell script) de vardır.

Üst seviye dil donanımdan soyutlama (hardware abstraction) ve taşınabilirlik sağlar. Bedeli yavaşlamadır.

İşletim sistemi tasarımı matematiksel bir problem değildir. Bilinen yaklaşımlar vardır. Uzun yaşayan sistemler zamanla iyileşir.

Katmanlı yaklaşım, mikroçekirdek ve modüller

Katmanlı yaklaşım: Katman 0 donanımdır. Her katman yalnızca alt katmandan hizmet alır, üstüne hizmet verir. Amaç, donanımın hemen üstündeki çekirdeği küçültmektir.

Mikroçekirdek: Çekirdekte yalnızca en temel hizmetler kalır:

- bellek yönetimi,
- CPU zamanlama,
- süreçler arası iletişim (IPC).

Dosya sistemi, aygıt sürücüleri ve diğer hizmetler kullanıcı modunda çalışır. Bunlar birbirleriyle çekirdek üzerinden mesajlaşır. Örnek: bir uygulama dosya sistemine bir mesaj gönderir; mesaj mikroçekirdekten geçer. Dosya sistemi diske erişirken de aygıt sürücüsüne yine mikroçekirdek üzerinden mesaj gönderir.

- Artısı: dosya sistemi kullanıcı modunda olduğu için kullanıcılar birbirinden yalıtılabilir. Konteyner (container) teknolojisinde bu fikir kullanılıyor. Hoca konteynerleri, dağıtık sistemleri ve sanallaştırmayı ileride ayrıca konuşacaklarını söyledi.
- Eksisi: her mesaj kullanıcı modundan çekirdek moduna, sonra tersine bir geçiş ister. Bu ek yük performansı düşürür.

Örnek: Mach mikroçekirdeği. Mac OS X bu yapı üzerine kurulmuştur, ancak bugünkü sürümler modüler ve karma bir yapıdadır.

Yüklenebilir çekirdek modülleri: Bugünkü Linux sistemleri bunu kullanır. Bir çekirdek parçası çekirdeğe gömülü olabilir ya da modül olarak yazılabilir. Modül, modül arayüzü üzerinden çekirdekle konuşur.

- Gerektiğinde `insmod` ile yüklenir. `modprobe` bağımlı modülleri de otomatik yükler.
- İş bitince `rmmod` ile kaldırılır.
- Bellekte yalnızca o an gereken modüller durur.

Solaris örneğinde çekirdek ortadadır; dosya sistemleri, zamanlama sınıfları, aygıt sürücüleri vb. modüllere ayrılmıştır.

Karma yapılar:

- Linux: büyük ölçüde modüler, bir kısmı monolitik.
- Windows: büyük ölçüde monolitik; bazı kısımları mikroçekirdeğe benzer, modüller de var. Hoca, Windows 10 mimarisini çizmenin karmaşık olacağını söyledi.
- Mac OS X: katmanlı ve modüler. Bir tarafta BSD ortamı, diğer tarafta Mach var.

Süreç kavramı

Süreç (process), programın çalışan hâlidir. Program diskte duran komutlar bütünüdür. Program belleğe alınıp çalıştırılınca süreç olur.

Bir süreçte şunlar bulunur:

- **Text bölümü:** programın kodu.
- **Program sayacı:** bir sonraki çalışacak komutun adresi. Çok görevli sistemde süreç kenara alınıp geri konduğu için nerede kalındığı bilinmelidir.
- **Yazmaçlar:** süreç çalışırken sonuçlar yazmaçlarda oluşur. Örneğin bir döngünün sayacı CX yazmacında olabilir. Süreç geri yüklendiğinde yazmaç değeri farklıysa program yanlış çalışır. Bu yüzden yazmaç seti de saklanmalıdır.
- **Data bölümü:** global değişkenler.
- **Stack:** geçici bilgi. Fonksiyon parametreleri, dönüş adresi, yerel değişkenler.
- **Heap:** çalışma sırasında dinamik olarak ayrılan bellek (`malloc` vb.).
- Açık dosyalar ve kullanılan aygıtlar.

Text ve data boyutu sabittir. Stack ve heap boyutu değişir. Bu yüzden stack yukarıdan aşağı, heap data'nın hemen üstünden yukarı doğru büyür.

Resimde adresler 0'dan max'a kadardır. Süreç belleğin tamamı kendisininmiş gibi davranır. Gerçekte işletim sistemi ve diğer süreçler de bellektedir. İşletim sistemi süreci mantıksal (lo-

gical) adreslerle çalıştırır. Fiziksel bellekte yalnızca kullanılan segmentler parça parça yerleştirilir. Amaç belleği verimli kullanmaktır.

Süreç durumları

Temel durum diyagramında 5 durum vardır. Farklı işletim sistemlerinde küçük farklar olabilir.

Durum	Anlamı
new	Süreç yaratılıyor
ready	CPU dışında her şeyi hazır, çalışmayı bekliyor
running	CPU'da çalışıyor
waiting	Bir olayı (ör. G/Ç bitişi) bekliyor
terminated	Çalışması bitti

- Ready'deki süreçlerden birini seçip CPU'ya vermek zamanlamanın (scheduling) işidir.
- Running'deki süreç G/Ç başlatırsa waiting'e geçer. G/Ç denetleyicisi (ve varsa DMA) işi bitirip haber verene kadar CPU başka sürece verilir.
- Süreç normal ya da anormal biçimde bitebilir. İki durumda da terminated'a geçer.
- Terminated durumu, ebeveyn sürecin çocuğun sonucunu öğrenebilmesi için gereklidir. Örneğin kabuk, çalıştırdığı komutun başarılı olup olmadığını bu bilgiden öğrenir.

Süreç kontrol bloğu (PCB)

İşletim sistemi her süreç için bir bilgi yapısı tutar. Bu yapı C'deki `struct` gibidir. Linux'ta adı `task_struct`'tır. İçinde:

- süreç kimliği (PID),
- süreç durumu,
- program sayacı ve yazmaçlar,
- CPU zamanlama bilgisi (öncelik, son zaman diliminin ne kadarının kullanıldığı vb.),
- bellek yönetimi bilgisi,
- hesap bilgisi (verilen CPU zamanı vb.),
- G/Ç durum bilgisi (açık dosyalar, aygıtlar).

Bağlam değişimi örneği

Sistemde P0 ve P1 olsun.

1. P0 çalışıyor. Bir sistem çağrısı yapıyor (ör. klavyeden girdi okuma).
2. Girdi gelene kadar P0'ı tutmanın anlamı yok. İşletim sistemi P0'ın program sayacını, yazmaçlarını vb. P0'ın PCB'sine kaydeder.
3. P1'in PCB'sindeki bilgiler yüklenir. P1 kaldığı yerden devam eder.
4. P1, bir kesme (ör. zaman dilimi bitince zamanlayıcı kesmesi) ya da sistem çağrısı olana kadar çalışır.
5. Sonra P1'in durumu PCB'sine kaydedilir, P0'ınki yüklenir.

Bu işlemin tamamına bağlam değişimi (context switch) denir.

Bir öğrenci, iki süreç de beklerken ne olduğunu sordu. Hoca: CPU'ya güç ve saat sinyali verildiği sürece çalışır. Windows'ta bu durumda "System Idle Process" çalışır.

İş parçacıklarına giriş

Şimdiye kadar her süreçte tek bir akış vardı. Benzer işi yapan birden fazla akış gerekirse her biri için yeni süreç yaratmak pahalıdır: kod, data, heap ve stack yeniden kopyalanır.

İş parçacıkları (threads) kod, data ve heap'i paylaşır. Her iş parçacığının ayrı stack'i vardır. Stack ayrı olmalıdır, çünkü her akış kendi fonksiyon çağrılarını stack'e basar. Ortak stack olursa akışlar birbirinin bilgisini bozar.

Çok iş parçacıklı sistemde PCB'de iş parçacığı bilgileri de tutulmalıdır. Konu ileride ayrı bölümde işlenecek.

Zamanlama kuyrukları

Amaç CPU'yu verimli kullanmaktır: G/Ç başlatan süreci kenara alıp CPU'yu başkasına vermek, zaman paylaşımıyla herkesin biraz ilerlemesini sağlamak.

- **Job kuyruğu:** sistemdeki bütün süreçler.
- **Ready kuyruğu:** CPU dışında her şeyi hazır süreçler.
- **Aygıt (bekleme) kuyrukları:** her aygıtın ayrı kuyruğu. Süreç hangi aygıtı bekliyorsa onun kuyruğundadır.

Kuyruklar bağlı liste ile gerçeklenir. Böylece PCB'ler hızlıca eklenip çıkarılabilir.

Running'deki bir süreç şu durumlarda CPU'dan alınır:

- G/Ç isteği yaptı: ilgili G/Ç kuyruğuna gider.
- Zaman dilimi (time quantum) bitti: ready kuyruğuna döner.
- Çocuk süreç yarattı (`fork`): ebeveyn ready kuyruğuna döner, çocuk da ready kuyruğuna girer. Hangisinin önce çalışacağına zamanlayıcı karar verir.
- Bir kesmeyi bekliyor.

Kısa, uzun ve orta vadeli zamanlayıcı

- **Kısa vadeli (short-term, CPU scheduler):** bir sonraki CPU döngüsünde hangi sürecin çalışacağını seçer. "Zamanlayıcı" denince genellikle bu kastedilir. Derste de bu konuşulacak.
- **Uzun vadeli (long-term, job scheduler):** ikincil bellekteki (secondary storage) işlerden hangilerinin, kaçının belleğe alınacağını seçer.
  - Bugünkü masaüstü sistemlerde doğrudan böyle bir yapı yoktur. Hangi sürecin yaratılacağına kullanıcı ya da başka bir sistem karar verir.
  - Kavramın önemi denge kurmaktır. Hepsi G/Ç ağırlıklı olursa süreçler G/Ç kuyruğunda, sonra ready kuyruğunda trenler oluşturur. G/Ç ağırlıklı ve CPU ağırlıklı süreçler karışık olursa G/Ç ve CPU aynı anda kullanılır.
- **Orta vadeli (medium-term):** bellekte duran ama şu an çalışamayan (ör. uzun G/Ç bekleyen) bir süreci ikincil belleğe çıkarır (swap out), çalıştırılmak istenen bir süreci belleğe alır (swap in). Bu işe takas (swapping) denir.

Bir öğrenci RAM'den çıkarılan sürecin nereye gittiğini sordu. Cevap: ikincil belleğe (diske). İhtiyaç olunca yeniden RAM'e getirilir. Süreç çalışabilmek için ana bellekte olmalıdır.

Ders sonrası notlar

- **Altyazı hataları:** "SMB'li" aslında Assembly; "Algor, PL1" aslında ALGOL ve PL/I; "Mac kernel" aslında Mach çekirdeği; "Linux diye bir işletim sistemi var, real-time olma derdinde" cümlesinde işletim sisteminin adı anlaşılmıyor (gerçek zamanlı bir sistemden söz ediliyor).
- **Komut yorumlayıcısı:** 00:08:30 civarında hoca "resident kısım bellekten kaldırılıyor" dedi. Hemen ardından anlattığı süreç doğrudur: kaldırılan **geçici (transient)** kısımdır; program bitince **yerleşik (resident)** kısım geçici kısmı yeniden yükler.
- **Katmanlı yapı ve mikroçekirdek:** Derste ikisi aynı anlatım içinde geçti ve Mach katmanlı yapının örneği gibi sunuldu. Bunlar farklı yaklaşımlardır. Katmanlı yaklaşımda işletim sistemi katmanlara bölünür. Mikroçekirdekte çekirdek en aza indirilir, hizmetler kullanıcı modunda süreçler olarak çalışır. Mach bir mikroçekirdektir.
- **Mac OS X:** Mac OS X baştan beri Mach ile BSD'yi birleştiren karma bir çekirdek (XNU) kullanır. "Önce saf mikroçekirdeğe geçtiler, sonra modüle döndüler" diye okunmamalı.
- **Heap büyümesi:** Derste "segment dolunca yeni segment açılıyor" denildi. Bu bir basitleştirmedir. Gerçekte heap, bellek ayırıcının isteğiyle (`brk`/`sbrk` ya da `mmap`) büyütülür. Ayrıntı bellek yönetimi bölümünde.
- **fork sonrası:** Linux'ta süreçler ve iş parçacıkları aynı `task_struct` yapısıyla temsil edilir. Derste "fork bir süreç yaratıyor ama arkada bir thread var" denmesinin anlamı budur.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.